

W10- PRACTICE

FIREBASE (REST API)

Learning objectives

- ✓ Handling a **HTTP PUT** to update an existing data element
- ✓ Handling an **IN-MEMORY CACHE** on repositories
- ✓ Handling HTTP **POST** to add new data element

How to start?

- ✓ Copy the **START CODE** into your current repository and push it

W10 - Start Code

- ✓ Each task of this practice shall be related to commit(s) including the **tasks ID**.

W10 -01 – Fetch and display songs

How to submit?

- ✓ Once finished, submit on MS Team
 - Your GitHub repository URL
 - This document if needed



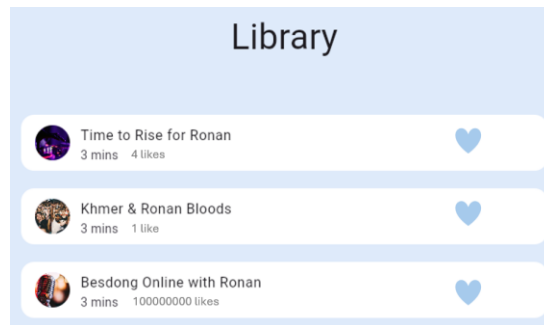
This practice focusses more **on clean code structure** rather than Flutter technical skills.
We will evaluate how well **you follow the coding conventions**, how you name your class, variables etc.

No code generated by AI is allowed

W10-01 – Handle the like-a-song button

We want to be able to like a song by clicking on below like buttons.

Note: we don't care about who liked the song (no user id needed), but just a simple counter.



? Firebase DB: How will you update your Firebase database to handle the number of likes?

- Each song should have a likes field, starting at 0. When the user presses the like button, your app updates that song's likes value by increasing it by 1

? HTTP Request: What kind of HTTP verb will you choose to increment the likes?

- Use "PATCH", because you only need to update one field (likes) of the song instead of replacing the whole song object

? Repository: Propose a method in the Song Repository to handle liking a song

- ✓ What should the **method parameters** be? (e.g., song ID, full Song object, current number of likes?)
- ✓ What should the **method return**? (e.g., Future<void> or Future<Song>?)
- ✓ What should be the possible errors?

- Parameter: songId is enough, because the repository can read the current likes and update it internally

- Return type: Future<Song> is best, so the UI gets the updated song (including new likes) immediately

- Possible errors: song not found (404), network/timeout error, permission from firebase deny, update conflict (if two users like at the same time), invalid server

? Model View:

- ✓ How will you update the model view AsyncValues ?
- ✓ What will you do in case of error?

- Set AsyncValue to loading when like starts, then set it to success with updated song data when like finishes

- Set AsyncValue to error, restore previous likes value if needed (rollback), and show an error message to the user (for example, Snackbar)

? View:

- ✓ What kind of screen or screen need to be updated?
- ✓ What will you do in case of error?

- Update the Library screen item UI, especially the like button and likes count text for each song
- Keep the old likes count, stop loading state, and show a short error message so the user knows the like action failed

W10-02 – Handle In-Memory Caches

We want to handle [in-memory caches](#) on each repositories

The repository sits between:

- The **data source** (API, DB)
- The **application logic / UI**

So it's the *perfect place* to:

- avoid duplicate API calls
- keep already-fetched data alive during app usage

A basic template to handle a cached value in the repository could be:

```
List<X>? _cachedXX;

Future<List<Song>> getXX() async {
  // 1. Return cache if available
  if (_cachedXX != null) {
    return _cachedXX!;
  }

  // 2. Otherwise fetch from API
  final xx = await _fetchXXs();

  // 3. Store in memory
  _cachedXX = xx;

  return songs;
}
```

Q1 - Implement caches in the different repositories (Song, Artist)

Q2 – Add a RefreshIndicator widget to force the refresh (clear the cache)

<https://api.flutter.dev/flutter/material/RefreshIndicator-class.html>

Note you can update the repository API with an option to force the fetch:

```
Future<List<Song>> getXX({bool forceFetch = false}) async
```

Q3 – We can still see a very fast tiny “loading flash”: can you explain why?

TIP: think about the view model lifecycle, when are we creating it?

=> The view model sets loading before every fetch, so even fast cached data shows a quick loading frame. And each tab rebuilds its own ChangeNotifierProvider, so the view model gets recreated and `_init` runs again, causing the brief flash.

W10-03 – Artist Screen with Songs and Comments



For this task, you are allowed to get some guidance with AI to code faster (if you want).

We want to create another screen for a specific artist

- Displays a list of songs for a given artist
- Allows the user to post a comment about the artist

1 Model Layer

- Create the Comment model
- Update the Artist model (e.g., add new fields if needed)

2 DTO / Repository Layer

- Update the Artist DTO (if necessary for songs/comments)
- Update or create the Artist Repository methods:
 - Fetch artist songs
 - Fetch artist comments
 - Post a new comment

3 ViewModel

- Create Artist VM
- Add state variables:
 - List<Song> songs
 - List<Comment> comments
- Implement fetchData() → fetch songs and comments
- Implement addComment() → post new comment, update local state, notify listeners

4 UI Layer — Screen & Widgets

- Create Artist Screen
- Re-use existing SongTile widget to display songs
- Create CommentTile widget to display comments
- Show placeholders for empty song or comment lists
- Create Comment form:
 - Text field + submit button
 - Placed in a bottom navigation panel or bottom sheet
 - Clear comment input after submitting a comment
 - Validate empty comments before posting

My Goal: W10-03 – Artist Screen with Songs and Comments

Build an Artist detail screen that:

- Displays songs of a selected artist
- Shows comments for that artist
- Allows users to post a new comment

1. Model Layer :

Added a new Comment model with: id, artistId, message, author, createdAt

- Updated Artist model to include optional extra info (about)

2. DTO and Repository Layer:

- Added CommentDto and updated ArtistDto
- Updated Repository now can fetch songs, fetch comments, and post comments
- Added simple caching for songs and comments per artist

3. ViewModel Layer:

- Created ArtistViewModel
- Keeps songs, comments, and loading/error state
- fetchData() loads everything
- addComment() posts and updates the list

4. UI Layer

- Created ArtistScreen
- Displays songs and comments
- Shows message if empty
- Added comment input (no empty submit, clears after send)
- Tap artist to open this screen

Architecture

W10-03 is implemented with a clean architecture:

- **Repository** -> handles data and caching
- **ViewModel** -> manages state and business logic
- **UI** -> handles rendering and user interaction